

1. Introduction

The following project is a first-person-POV Virtual Reality Maze Adventure game. This game is set in a fictional fantasy world, the Labyrinth of Nexos, created for this assignment. The player is transported into Nexos and must solve various mazes to leave.

The demographics for this VR game are hobby gamers and anyone who enjoys VR Adventure games and fast-paced problem-solving.

This game is split into four scenes: the main menu, level 1, level 2 and the exit menu. The main menu offers two options: play the game or exit. If they choose to play the game, they get placed into the first level. The first level is a simple maze layout with two types of help items to aid the player in solving it. At the end of level 1, a portal awaits the player, transporting them to level 2. Level 2 is a more complex maze with fewer help items. Again, at the end of this level, a portal awaits the player, transporting them to the exit menu. In the exit menu, the player can choose to start a new game or quit.

The user explores the game through a VR headset and two controllers. The controllers enable the user to interact with game objects and control their movements.

The game was made in Unity and assets from the Unity Asset Store, and Presets from Unity's VR Sample scene were used to create this game.

Link to the playthrough of the game: <https://youtu.be/fIH-nNG93LQ>

2. Resources

A VR game was selected over a mobile format because I believed a VR environment would best immerse the player in the game's fictional world. Furthermore, I felt that a VR game would offer the best gaming experience for a maze adventure. However, testing played a crucial role in this process. Initially, I borrowed an Oculus Quest 2 from the library to test the game. My original plan was to use the Meta Link functionality of the Quest 2, which allows the headset to connect to a computer and enable real-time game view updates in Unity. However, I soon realised that the Meta Link app was Windows-only and required a GPU, neither of which I possessed. Since I still had the Oculus Quest 2, I decided to build the game for Android. Using Sidequest, I side-loaded this APK onto the Quest 2, allowing me to thoroughly test the game with a VR headset. Although it wasn't real-time testing like Meta Link would have provided, it offered richer insights by allowing direct testing on the end device. As iPhones do not permit sideloading of APKs, and I lacked access to an Android device, I used the XR Device simulator to test the project. This tool translates mouse and keyboard input to control simulated XR controllers and an XR head-mounted display (HMD). It was an invaluable resource, enabling me to test basic game mechanics like movement and mouse and keyboard interactions without having to build the full game and export it to the Oculus Quest 2.

Initially, I planned to create a 3D third-person POV maze game inspired by maingazuntype's third-person maze adventure. I had already implemented the third-person tracking camera. After developing this and learning more about VR through the lectures, I decided to shift from a third-person POV to a first-person VR experience. Through this exploration, I gained many insights into how the maze could shape the game's genre and more. I also studied the dynamics and techniques of camera manipulation, focusing on the player character. This helped me explore how the camera should follow the character to enable exploration without revealing too many of the maze's secrets and to preserve its challenges.

These camera techniques informed the VR device's first-person perspective and how the camera would follow an in-game sparrow as it explores the maze, aiding the player as a utility.

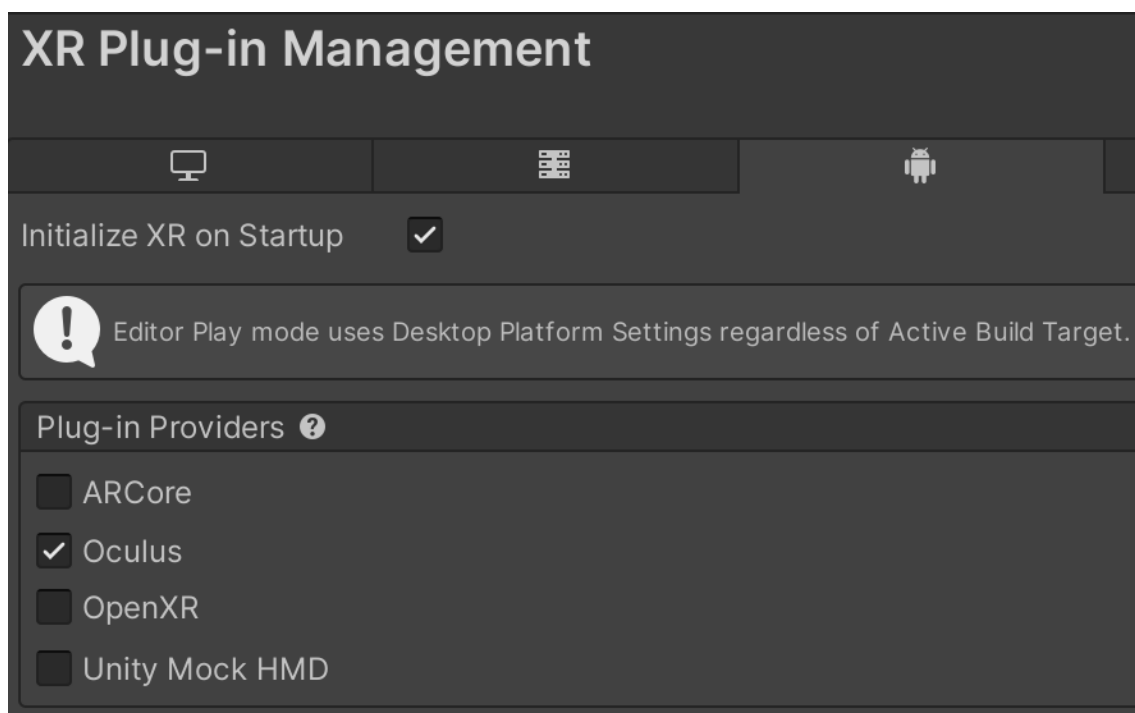
To build my world, I utilised a variety of Unity Asset Store assets, which I have linked below. Using these assets and presets allowed me to create a comprehensive world that immerses the player without requiring excessive time investment. This enabled me to focus my time on this project more strategically and productively. These assets also helped me learn more about Unity and incorporate more challenging elements, like the portal animations, that I otherwise would not have been able to include or create myself.

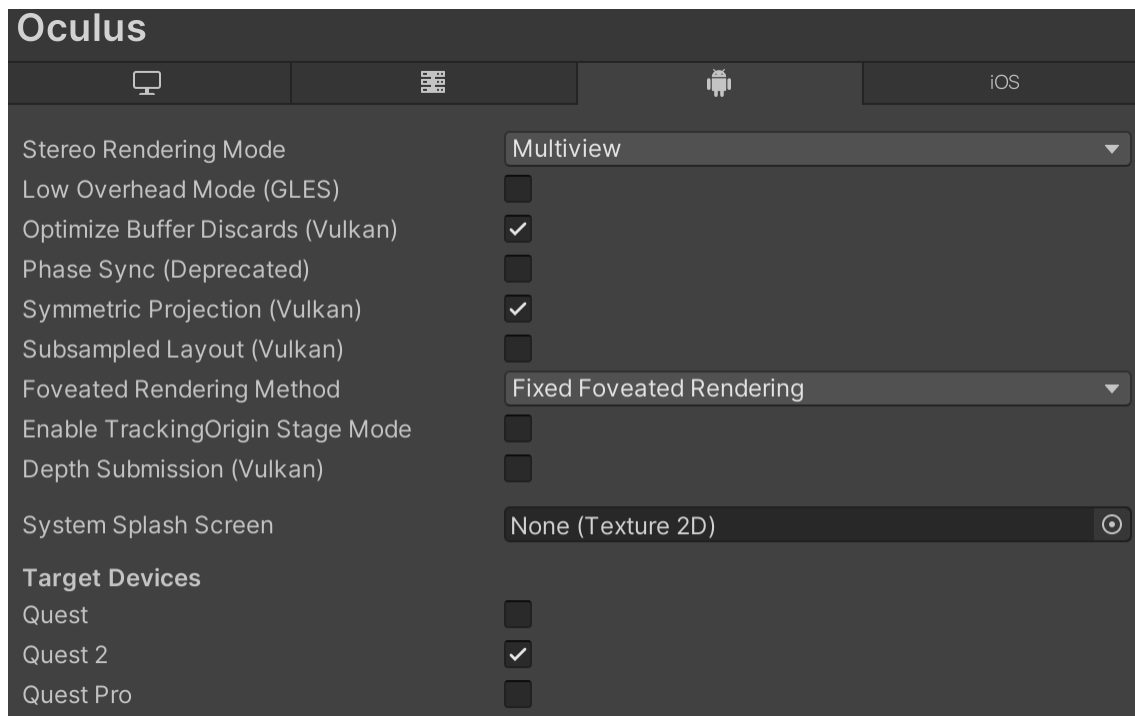
3. Process

The following section was split between the report and the demo video. The demo video can be accessed [here](#).

XR Interaction Toolkit

When building a VR game with Unity, the XR Interaction Toolkit is the most commonly used framework. This is a high-level, component-based interaction system for creating VR and AR experiences in Unity. It supports input from multiple XR controller inputs. For this project, I used the Oculus configuration.



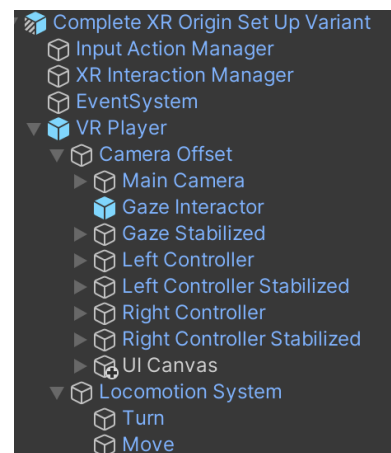
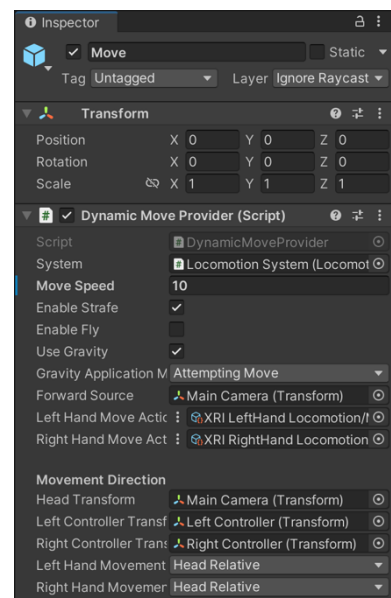


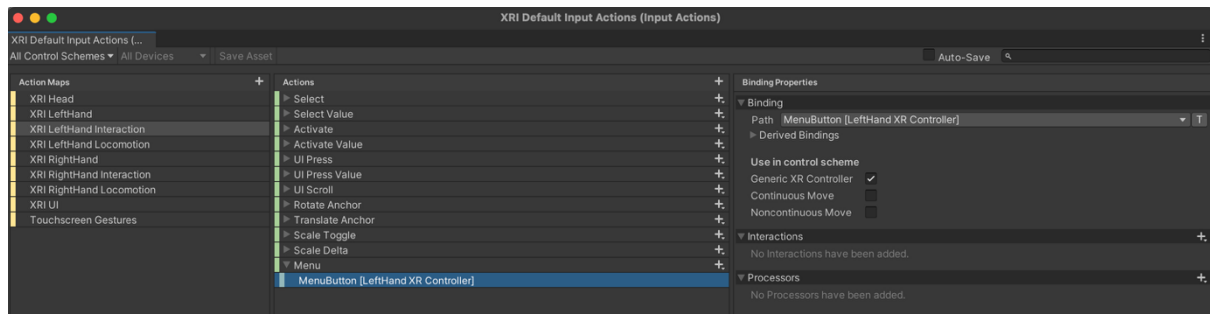
This allowed the game to successfully use input from a Quest 2 headset and controllers to control its inputs and actions. Within the scenes, the XR Origin Set Up Variant Prefab is used. This provides the camera that the headset tracks, as well as the models for the two controllers that follow the user's hands. The VR controller utilises a Character Controller component that enables collisions with any Collider Object. This stops the player from walking through walls and triggering elements such as the tooltips and the portal.

A Locomotion System controls the VR player based on the player's inputs and movement.

The main movement types in the VR game are Turn and Move. A 45-degree turn feels most natural and is industry standard for VR. Move controls the user's in-game movements, using a speed of 10 and standard gravity for the most natural, realistic feel in the game environment.

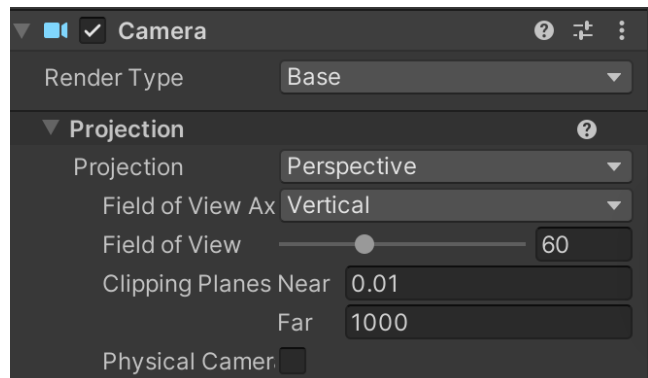
The Locomotion system links movements to components and transforms, using the main camera as its forward source to determine which direction the player is facing and which inputs the user needs to trigger this action. These are defined in the Input Action Manager. Mapping the players' inputs to actions within the game is essential for a playable experience. This defines the role of the Input Action Manager. In the Input Action Manager, controller buttons can be mapped to game actions. In the VR Game, the meta menu button toggles the in-game menu. The defaults were used for the remaining control mappings to provide a familiar experience for the user.





Camera

As mentioned before, the XR Origin Prefab already contains a camera. This camera gives the player a view of the game. Settings such as Field of View (FOV) and Clipping planes were utilised to provide a highly usable experience for the user. The FOV determines how much of the world the player sees at any given moment. A FOV of 60 was used to balance visibility and power drawn. Clipping planes determine how far an object is shown to the player. Reducing the Far Clipping planes improved performance as less needed to be rendered.



A balance was needed to ensure the best gaming experience, emphasising natural eye movement to reduce motion sickness, a common VR side effect. An essential part of the VR game is the controllers. They act as a bridge between the real world and the VR environment. With the controllers, the player can explore the maze and interact with the surroundings. The user can press the grab button on the controller to interact with objects, such as the Sparrow. The user receives immediate feedback if they are in range of the sparrow, as the controller ray changes colour from white to blue.

Maze

To create the maze, a plane was designed, and its material was generated from a maze layout image. To achieve this, a JPEG image of the desired maze layout was imported into Unity and then linked to the Base Map component in the plane's shader. Both maze layouts are included in the appendix. Cubes were used to form the maze walls by scaling them appropriately.

To design the levels, two distinct maze layouts were used. It was essential to ensure the second level was more challenging than the first, as increasing difficulty keeps players engaged and provides a sense of progression. This was accomplished in two ways. Firstly, fewer help items were included in the second level. The second level only has signs to guide the player, whereas in the first level, the player gets an aerial view. Secondly, the second level features a more intricate layout. In the first level, the player isn't introduced to frequent dead ends. Most incorrect paths are short and clearly identifiable before entering. This was changed for the second level. The player is presented with more choices for navigating through the maze.



Figure 1 - Maze Layout Level 1

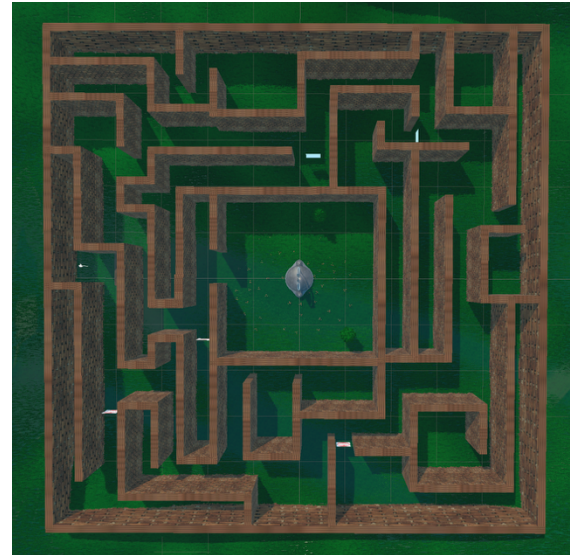


Figure 2 - Maze Layout Level 2



The Do Not Enter Sign was imported from US Road Signs Free asset, dragged into the scene, then linked to the right texture for display.

For the wall materials, a Unity Asset from Phoenix3D Studio, P3D: Outdoor Wall Tile Texture Pack (LR), was used. The textures were connected to the Surface Inputs in the Shader, and the tiling was adjusted to achieve the best result across all walls.

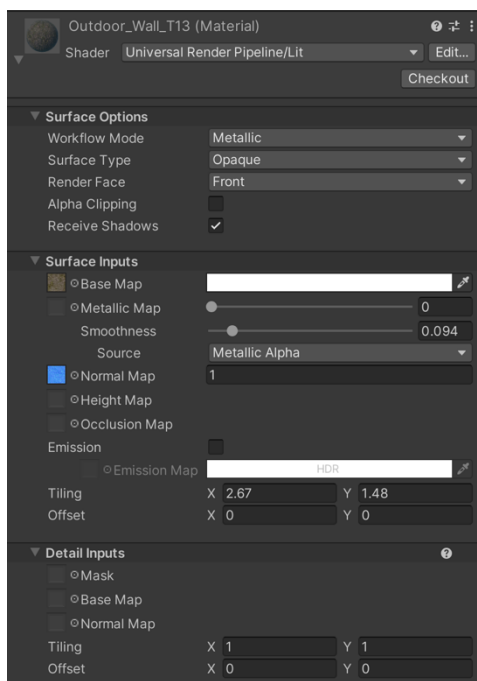


Figure 3 - Walls in Level 1

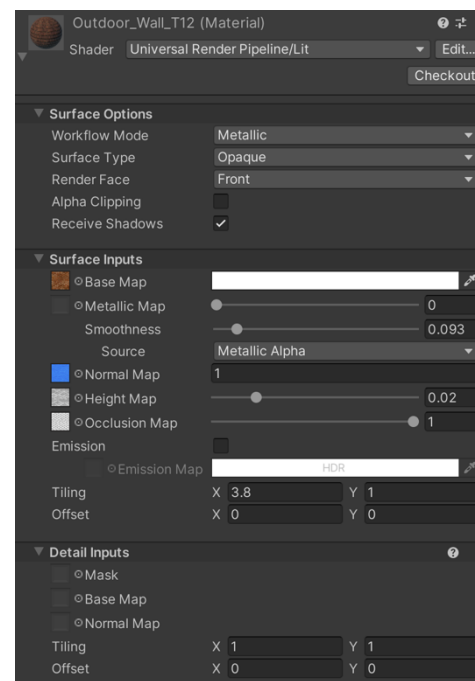


Figure 4 - Walls in Level 2

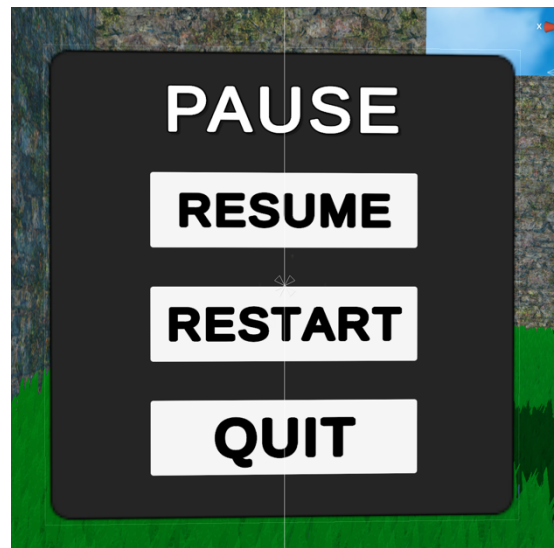
Wooden Board

A noticeboard was added at each level's start to improve storytelling and guide players. The Woodenboard, based on a Unity Asset from Logicalbeat, was imported and placed in the scene. It was simplified by using only one parchment to minimise clutter. Initial overlay text was messy and didn't match the scene's style, so tooltips were created instead.



In Game Menu

The In-Game was created by adding a UI Canvas to the scene and setting the render mode to World Space. Then a panel was parented to the canvas, and three buttons were made. The menu is toggled by the left Meta button on the player's controller, and the resume button toggles the UI canvas again so the player can return to the game. The restart button reloads the current level, and the quit button exits the application. The restart uses SceneManager to load the scene, while Quit uses Application.Quit. All buttons' functions are set in the button component, with OnClick linking scripts to game objects.



The user interacts with buttons by hovering with their VR controller and pressing the grab button. This creates an intuitive selection system. In the XR Device Simulator, buttons respond to mouse clicks. The UI canvas was attached to the XR Origin VR Player prefab to ensure that it is always in view. The UI canvas was attached to the XR Origin VR Player prefab to ensure that it is always in view.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Used to toggle In game menu depending on input from the left Meta Menu Button on the controller
public class ToggleMenu : MonoBehaviour
{
    public GameObject target;

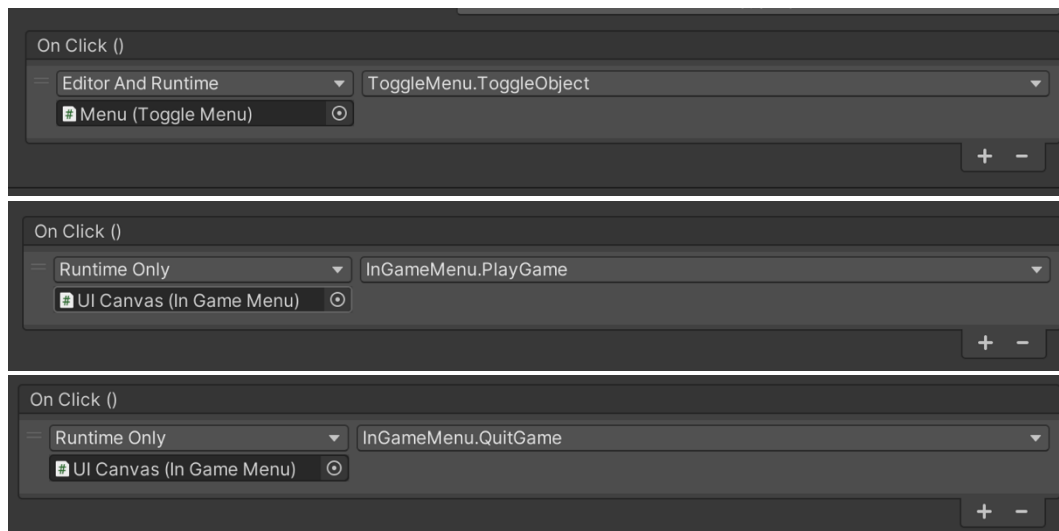
    public void ToggleObject()
    {
        // Sets the object to be opposite of what it currently is
        target.SetActive(!target.activeSelf);
    }
}
```

```
// Actions for each button of In Game Menu
public class InGameMenu : MonoBehaviour
{
    public void PlayGame()
    {
        // Restarts the current level

        SceneManager.LoadSceneAsync(SceneManager.GetActiveScene().buildIndex);

    }

    public void QuitGame()
    {
        // Quits the game application
        Application.Quit();
    }
}
```



Main & Exit Menu

The Main Menu is the first scene the player sees when opening the game, and the exit menu is the last scene the player sees.

The Main Menu introduces the player to the gaming world and gives them the option to play or quit the game. These menus were created the same way as the In-Game Menu and use the same functions, with the only difference being that the play function always loads level 1. Similar to the in-game menu, the user can interact with the Main and Exit menu either with their VR Controller. The ray from the in-game controller goes blue when the user is able to select their choice, which provides clear feedback on selecting the buttons.

The UI canvas was again attached to the XR Origin VR Player prefab to ensure that it is always in view.



Sound

Three audio files were used within the game. Each level has its own game sound. For this, the Unity Asset, Free 10 Medieval Ambient Fantasy Tracks Music Pack, by Alkakrab, was used. In both levels, the Audio Source component was added to the Ground Games Object.

Additionally, sound was added to the Sparrow preset to improve the game experience. Additionally, a script was created so the sound would play only when the player enters the trigger collider of the Sparrow.

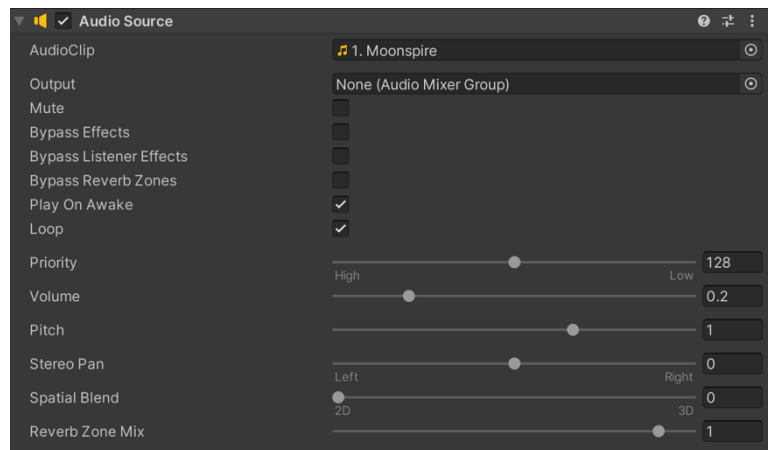


Figure 5 - Audio for Level 1

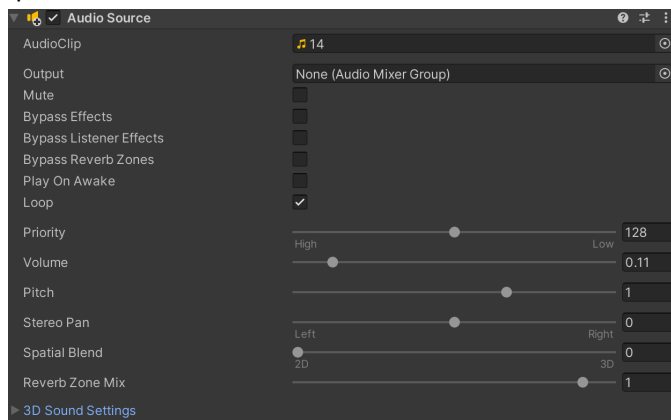
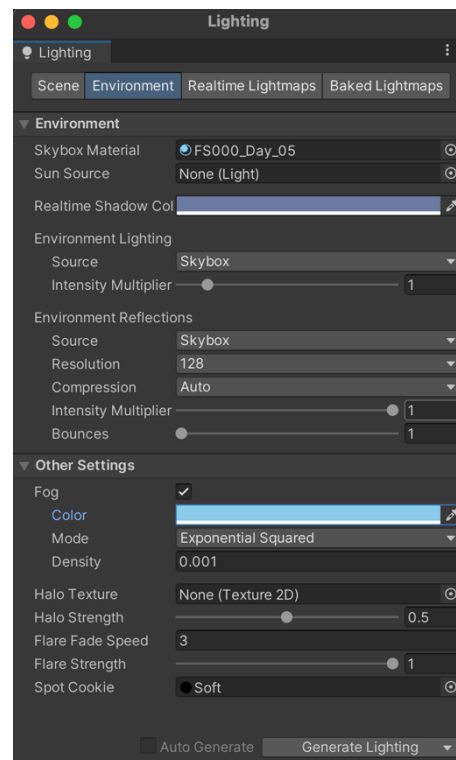
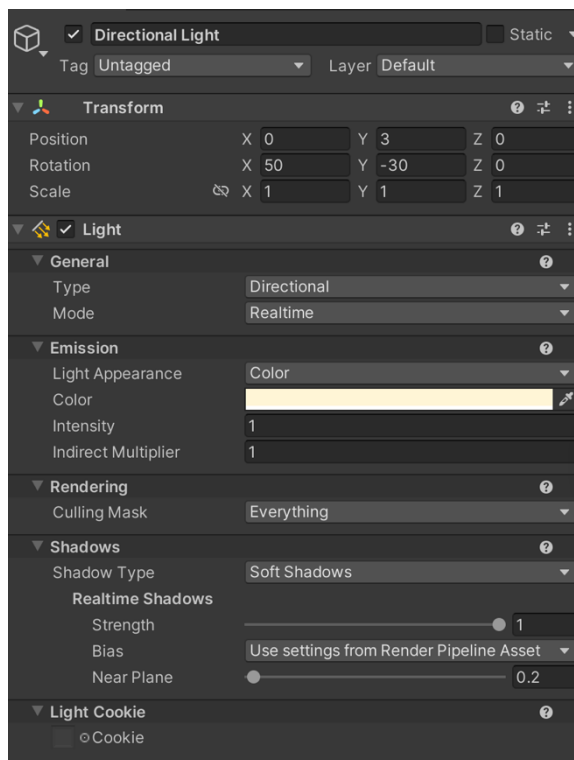


Figure 6 - Audio for the Sparrow

Lighting & Environment

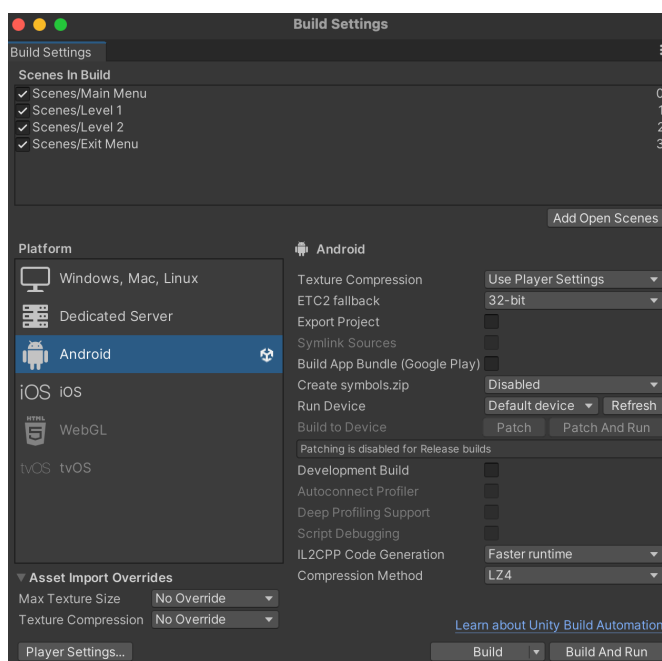
The default scene lighting was used and required no adjustments.

For the sky, the Unity Asset Fantasy Skybox FREE was used and connected to the Skybox Material tab in Environment settings. Fog was added to levels 1 and 2 using the sky's lightest colour, with density adjusted for natural appearance.



Build Settings

The game is designed for Android and intended to run on a Meta Quest 2. The default build settings were used, and the four scenes included in the game were added to the build. The game is then compiled into an APK, which can be sideloaded onto the Quest 2 via Sidequest.



The Terrain, Tooltips, Sparrow, and Portal are discussed in the Demo Video, available [here](#).

4. Reflection

In my opinion, the project turned out great. Although there are certain areas I wish I could have improved, overall, I am pleased with the outcome. The game's navigation and narration structure worked effectively, and the materials, sounds, and effects used suited the scene well. However, because the tiling is unified across all materials applied to game objects, scaling the walls distorted the material. Another aspect I wish I could have addressed is the terrain's limited loading distance. Due to lag, I had to reduce the loading distance for the grass texture and trees. Not doing so would have resulted in an almost stop-motion game that could cause motion sickness while navigating the maze. Lastly, I wish I had utilised prefabs more throughout the whole game. I ended up copying and pasting many elements across scenes. Having created prefabs previously would have made this process much easier. If I were to make this game again, I would spend more time creating a rich world for the player to explore while solving the maze, and I would incorporate more helpful items that interact with the maze, such as objects that move or break maze walls.

5. Appendix

The following tutorials helped create this game. Additionally, I used the listed Assets from the Unity Asset Store within my game. The game objects, scripts or scenes recreated from the tutorials were customised after watching the tutorials and understanding the process.

Demo Video: <https://youtu.be/vkS0bkV2X6E>

Playthrough Video: <https://youtu.be/fIH-nNG93LQ>

Game Inspiration:

https://www.reddit.com/r/IndieGaming/comments/1drpi1j/im_building_a_thirdperson_maze_adventure_and_im/

Tutorials:

- Spaderdabomb "Build a beautiful 3D open world in 5 minutes | Unity"

https://youtu.be/nCDGjLRecls?si=bmy9_3_GxLWrS281

- Codynn "How to Make a 3D Maze Game in Unity | Unity 3D Maze Unity Game Tutorial for Beginners | Part 7"

<https://youtu.be/fhRHnLwyKHE?si=U6eJQ702mOBQJKXm>

- Valem Tutorials "How to Make a VR Game in Unity 6.2 - Part 1 : Setup and Player"

<https://youtu.be/exc-73Mna3A?si=HgGIQmub0w52SOsB>

- Rehope Games "Make Your MAIN MENU Quickly! | Unity UI Tutorial For Beginners"

<https://youtu.be/DX7HyN7oJjE?si=9bY0u-Duvw7vcKWB>

- XR Terra Class "VR Player Collision in Unity"

<https://youtu.be/RMI2HqbmE74?si=BlMb8C5fKmrliZ6D>

- Omogonix "How to Change Scenes in Unity Using C# (Unity Tutorial 2021)"

https://youtu.be/gtpXc_9MR6g?si=QEeJ-4MtQ9M0rJ-q

- Fist Full of Shrimp "Mapping Inputs to Actions in Unity VR!"

<https://youtu.be/NObwdF9RqCg?si=V2d2XoQmJdfuKWDX>

- Valem Tutorials "How To Make a Tooltip for VR – Unity Tutorial"

https://youtu.be/xyoswWxj_6o?si=h7LOL89AmlZH_uID

- Rehope Games "How to Create a PAUSE MENU in Unity ! | UI Design Tutorial"

https://youtu.be/MNUYe0PWNNs?si=qVQfkuv_GBtyF1ff

- Bliz Studio "Unity VR XR Interaction Toolkit - UI Menu and Interactions"

https://youtu.be/Qb_8POkFwlk?si=A1hkPto52cmbX-P6

Assets from the Unity Asset Store:

Portal:

<https://assetstore.unity.com/packages/3d/environments/fantasy/runesandportals-85098>

Wall Texture:

<https://assetstore.unity.com/packages/2d/textures-materials/brick/p3d-outdoor-wall-tile-texture-pack-lr-247739>

Sparrow:

<https://assetstore.unity.com/packages/3d/characters/animals/sparrow-quirky-series-247228>

Woodenboard:

<https://assetstore.unity.com/packages/3d/props/woodenboard-292025>

Do not enter sign:

<https://assetstore.unity.com/packages/3d/props/exterior/us-road-signs-free-164941>

Skybox:

<https://assetstore.unity.com/packages/2d/textures-materials/sky/fantasy-skybox-free-18353>

Terrain:

<https://assetstore.unity.com/packages/3d/environments/lowpoly-environment-nature-free-medieval-fantasy-series-187052>

Audio:

<https://assetstore.unity.com/packages/audio/music/free-10-medieval-ambient-fantasy-tracks-music-pack-310781>

Maze Layouts:

Level 1:

<https://www.shutterstock.com/image-vector/maze-4-three-possible-outcomes-one-1252577014>

Level 2:

<https://www.vecteezy.com/vector-art/3622335-abstract-square-maze-with-entrance-and-exit-an-interesting-and-useful-game-for-children-simple-flat-vector-illustration-isolated-on-white-background-with-a-place-for-your-drawings>

XR Simulator-related Code:

```
using UnityEngine;
using UnityEngine.InputSystem;

[RequireComponent(typeof(CharacterController))]

// Used to override xr device simulator to use Character control move, this enables collisions in the
// simulator
public class CharacterControllerMovement : MonoBehaviour
{
    [Header("Input Actions (From Simulator)")]
    public InputActionReference keyboardXTranslate;
    public InputActionReference keyboardYTranslate;
    public InputActionReference keyboardZTranslate;

    // Transform of Main Camera
    public Transform headTransform;

    [Header("Movement Settings")]
    public float movementSpeed = 3.0f;
    public float gravity = -9.81f;
    // Character controller of Vr Player
    private CharacterController _controller;
    private Vector3 _velocity;

    void Start()
    {
        _controller = GetComponent<CharacterController>();
        // Enable keyboard X,Y,Z on start

        if (keyboardXTranslate != null) keyboardXTranslate.action.Enable();
        if (keyboardYTranslate != null) keyboardYTranslate.action.Enable();
        if (keyboardZTranslate != null) keyboardZTranslate.action.Enable();
    }

    void Update()
    {
        // return early if not enabled or null
        if (!_controller.enabled || headTransform == null) return;
        // Get current Keyboard Inputs
        float xInput = keyboardXTranslate.action.ReadValue<float>();
        float zInput = keyboardZTranslate.action.ReadValue<float>();

        // get Direction of where player is facing
        Vector3 headForward = headTransform.forward;
        headForward.y = 0;
        headForward.Normalize();

        Vector3 headRight = headTransform.right;
        headRight.y = 0;
        headRight.Normalize();

        // Move Player in direction they are facing
        Vector3 horizontalMove = (headForward * zInput) + (headRight * xInput);
        _controller.Move(horizontalMove * movementSpeed * Time.deltaTime);

        // Get velocity of player if they are falling or on ground
        if (_controller.isGrounded)
        {
            _velocity.y = -2f;
        }
        else
        {
            _velocity.y += gravity * Time.deltaTime;
        }
        // Move player with calculated velocity
        _controller.Move(_velocity * Time.deltaTime);
    }
}
```

Tooltip-related Code:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Used to rotate tooltips so it is always facing the VR player
public class VerticalBillboard : MonoBehaviour
{
    // Main Camera of the VR player
    public Transform target;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        // Make tooltip rotate to the VR player to always be front on
        transform.LookAt(target, Vector3.up);
    }
}
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[ExecuteInEditMode]

// Used to draw line between two points for tooltips

public class TwoPointLine : MonoBehaviour
{
    // Points to draw line between
    public Transform pointA;
    public Transform pointB;
    private LineRenderer Line;
    // Start is called before the first frame update
    void Start()
    {
        Line = GetComponent<LineRenderer>();
    }

    // Update is called once per frame
    void Update()
    {
        // Draw line between point a and point b
        Line.positionCount = 2;
        Line.SetPosition(0, pointA.position);
        Line.SetPosition(1, pointB.position);
    }
}
```

```
using UnityEngine;
// Used to enable tooltips when player is within trigger field
public class TriggerActivator : MonoBehaviour
{
    [Tooltip("The GameObject (or empty parent) that should be enabled when the player enters the trigger field")]
    public GameObject objectToActivate;

    [Tooltip("Tag assigned to the VR Player object.")]
    public string playerTag = "Player";

    [Tooltip("If true, the object will be deactivated again when the player leaves.")]
    public bool deactivateOnExit = true;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag(playerTag))
        {
            if (objectToActivate != null)
            {
                // Activate the object as player is in range
                objectToActivate.SetActive(true);
                Debug.Log($"Activated {objectToActivate.name}");
            }
        }
    }

    private void OnTriggerExit(Collider other)
    {
        if (deactivateOnExit && other.CompareTag(playerTag))
        {
            if (objectToActivate != null)
            {
                // Deactivate object as player has left range
                objectToActivate.SetActive(false);
                Debug.Log($"Deactivated {objectToActivate.name}");
            }
        }
    }
}
```

In-Game Menu-related Code:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Used to toggle In game menu depending on input from the left Meta Menu Button on the controller
public class ToggleMenu : MonoBehaviour
{
    public GameObject target;

    public void ToggleObject()
    {
        // Sets the object to be opposite of what it currently is
        target.SetActive(!target.activeSelf);
    }
}
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;

// Actions for each button of In Game Menu
public class InGameMenu : MonoBehaviour
{
    public void PlayGame()
    {
        // Restarts the current level
        SceneManager.LoadSceneAsync(SceneManager.GetActiveScene().buildIndex);
    }

    public void QuitGame()
    {
        // Quits the game application
        Application.Quit();
    }

    public GameObject menuPanel;
    public InputActionReference openMenuAction;

    private void Awake()
    {
        // Add ToggleMenu to actions on awake
        openMenuAction.action.Enable();
        openMenuAction.action.performed += ToggleMenu;
        InputSystem.onDeviceChange += onDeviceChange;
    }

    private void OnDestroy()
    {
        // Remove Toggle menu from actions on destroy
        openMenuAction.action.Disable();
        openMenuAction.action.performed -= ToggleMenu;
        InputSystem.onDeviceChange -= onDeviceChange;
    }

    private void ToggleMenu(InputAction.CallbackContext context)
    {
        // Toggle menu on call
        menuPanel.SetActive(!menuPanel.activeSelf);
    }

    private void onDeviceChange(InputDevice device, InputDeviceChange change)
    {
        // Required to handle in case of device disconnect
        switch (change)
        {
            case InputDeviceChange.Disconnected:
                openMenuAction.action.Disable();
                openMenuAction.action.performed -= ToggleMenu;
                break;
            case InputDeviceChange.Reconnected:
                openMenuAction.action.Enable();
                openMenuAction.action.performed += ToggleMenu;
                break;
        }
    }
}
```

Main Menu-related Code:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;

public class MainMenu : MonoBehaviour
{
    public void PlayGame()
    {
        // Loads the first level
        SceneManager.LoadScene(1);
    }

    public void QuitGame()
    {
        // Quits the game application
        Application.Quit();
    }
}
```

Portal-related Code:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

// Used to check when portal is triggered
public class PortalCollision : MonoBehaviour
{
    public string sceneName;
    // Trigger when entering collider
    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            // Load specified scene depending on what level the player is currently in
            SceneManager.LoadScene(sceneName);
        }
    }
}
```

Sparrow-related Code:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Used to allow vr player to follow bird when flying after interaction
public class FollowBirdVR : MonoBehaviour
{
    // Bird to follow
    public Transform bird;

    // Where bird is flying too
    public Transform pointB;

    // How fast to follow
    public float followSpeed = 3f;

    // Where to return player back to

    Vector3 startPos;
    bool following = false;
    bool returning = false;

    void Start()
    {
        // get the players position
        startPos = transform.position;
    }

    public void BeginFollowing()
    {
        following = true;
        returning = false;
        Debug.Log("Begin Following");
    }

    void Update()
    {
        if (following)
        {
            Debug.Log("Following");
            Vector3 target = bird.position;
            target.y = transform.position.y;

            // Move towards the Target position
            transform.position = Vector3.MoveTowards(
                transform.position,
                target,
                followSpeed * Time.deltaTime
            );

            // Triggers when at target position
            if (Vector3.Distance(bird.position, pointB.position) <
0.1f)
            {
                following = false;
                returning = true;
            }
        }

        if (returning)
        {
            // Return back to return
            transform.position = Vector3.Lerp(
                transform.position,
                startPos,
                followSpeed * Time.deltaTime
            );

            // Exit at starting position again
            if (Vector3.Distance(transform.position, startPos) < 0.1f)
                returning = false;
        }
    }
}
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Used to trigger the bird noise only when the player is within the collider
public class birdNoise : MonoBehaviour
{
    // Audio source of bird
    public AudioSource audioSource;

    private void OnTriggerEnter(Collider other)
    {
        // Only triggers on player collider
        if (other.CompareTag("Player"))
        {
            // Check if audio is already playing
            if (!audioSource.isPlaying)
                audioSource.Play();
        }
    }

    private void OnTriggerExit(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            // Turn off audio when player leaves
            audioSource.Stop();
        }
    }
}
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine.XR.Interaction.Toolkit;
using UnityEngine.XR.Interaction.Toolkit.Samples.StarterAssets;
using UnityEngine;

public class BirdFlight : MonoBehaviour
{
    // Determines the points the sparrow flies too
    public Transform pointB;
    public Transform midPoint;

    // Speed to fly
    public float speed = 2f;

    // Audio Source of Bird
    public AudioSource audioSource;

    // Transform VR Player
    public Transform xrOrigin;

    // Animator of Bird
    Animator anim;

    // Intractable Bird to disable when flying
    XRSimpleInteractable interactable;

    // Move provider to disable when flying
    public DynamicMoveProvider xrDynamicMoveProvider;

    // Parent to return Vr Player to
    Transform xrOriginalParent;

    // Original Location & Rotation of VR Player
    Vector3 xrStartPos;
    Quaternion xrStartRot;

    // Get Character Controller, rigid body and capsule to later disable
    CharacterController xrCC;
    CapsuleCollider xRCapsule;
    Rigidbody xrRb;
    CharacterControllerMovement xrMovement;

    // Sizes for character controller
    float ccHeight;
    float ccRadius;
    float ccStepOffset;
    Vector3 ccCenter;
    bool hasFlown = false;

    void Awake()
    {
        // get all the components required to control
        anim = GetComponent<Animator>();
        interactable = GetComponent<XRSimpleInteractable>();

        if (xrOrigin != null)
        {
            xrCC = xrOrigin.GetComponent<CharacterController>();
            xRCapsule = xrOrigin.GetComponent<CapsuleCollider>();
            xrRb = xrOrigin.GetComponent<Rigidbody>();
            xrMovement = xrOrigin.GetComponent<CharacterControllerMovement>();

            if (xrCC != null)
            {
                ccHeight = xrCC.height;
                ccRadius = xrCC.radius;
                ccStepOffset = xrCC.stepOffset;
                ccCenter = xrCC.center;
            }

            Debug.Log($"BirdFlight Awake - Found components: CC={xrCC != null}, Movement={xrMovement != null}, DynamicMoveProvider={xrDynamicMoveProvider != null}");
        }
    }
}
```

```

public void Fly()
{
    // Doesn't allow fly to be called twice, as it exits early if it has been done
    if (hasFlown)
        return;

    hasFlown = true;
    if (interactable != null)
    {
        interactable.enabled = false;
    }
    Debug.Log("=== FLY STARTED ===");

    if (xrOrigin != null)
    {
        // Get Original Values of XR before changes
        xrOriginalParent = xrOrigin.parent;
        xrStartPos = xrOrigin.position;
        xrStartRot = xrOrigin.rotation;

        Debug.Log($"Saved state - Pos: {xrStartPos}, Parent: {xrOriginalParent?.name ?? "null"}");

        // Turn off collisions to avoid issues with clipping
        DisablePlayerCollisions();

        // Set the XR to be a child of the bird
        xrOrigin.SetParent(transform, worldPositionStays: true);
    }

    if (anim != null)
    {
        // Trigger the fly activation
        anim.SetTrigger("Fly");

        // Start the fly routine
        StartCoroutine(FlyRoutine());
    }

    System.Collections.IEnumerator FlyRoutine()
    {
        if (midPoint != null)
        {
            // Fly first to midpoint usually up out of maze
            transform.LookAt(midPoint.position);
            while (Vector3.Distance(transform.position, midPoint.position) > 0.05f)
            {
                transform.position = Vector3.MoveTowards(
                    transform.position,
                    midPoint.position,
                    speed * Time.deltaTime
                );

                // waits a frame before proceeding
                yield return null;
            }
            Debug.Log("Reached midpoint");
        }

        // Turn to look at the Targe Position
        transform.LookAt(pointB.position);

        // Fly towards the target position
        while (Vector3.Distance(transform.position, pointB.position) > 0.05f)
        {
            transform.position = Vector3.MoveTowards(
                transform.position,
                pointB.position,
                speed * Time.deltaTime
            );
            yield return null;
        }

        Debug.Log("Reached final destination - Starting restoration");

        if (xrOrigin != null)
        {
            // Unparent from bird
            xrOrigin.SetParent(xrOriginalParent, worldPositionStays: true);
            Debug.Log($"Un-parented XR Origin, parent is now: {xrOrigin.parent?.name ?? "null"}");

            yield return null;

            // Teleport back to starting position
            xrOrigin.position = xrStartPos + Vector3.up * 0.8f;
            xrOrigin.rotation = xrStartRot;
            Debug.Log($"Teleported to: {xrStartPos + Vector3.up * 0.8f}");

            yield return null;

            // Reenable player collisions
            EnablePlayerCollisions();

            if (audioSource.isPlaying)
            {
                // Stop playing audio if it is still playing
                audioSource.Stop();
            }

            yield return null;

            Debug.Log("=== FLY COMPLETE - All systems re-enabled ===");
        }
    }
}

```

```
void DisablePlayerCollisions()
{
    Debug.Log("--- DISABLING Player Systems ---");

    // Disable XR Movement, MoveProvider, CharacterController, Capsule, Rigid Body
    if (xrMovement != null)
    {
        xrMovement.enabled = false;
        Debug.Log($"Movement script disabled: {!xrMovement.enabled}");
    }

    if (xrDynamicMoveProvider != null)
    {
        xrDynamicMoveProvider.enabled = false;
        Debug.Log($"DynamicMoveProvider disabled: {!xrDynamicMoveProvider.enabled}");
    }
    else
    {
        Debug.Log("DynamicMoveProvider not found");
    }

    if (xrCC != null)
    {
        xrCC.enabled = false;
        Debug.Log($"CharacterController disabled: {!xrCC.enabled}");
    }

    if (xCapsule != null)
    {
        xCapsule.enabled = false;
        Debug.Log($"CapsuleCollider disabled: {!xCapsule.enabled}");
    }

    if (xRb != null)
    {
        xRb.detectCollisions = false;
        Debug.Log($"Rigidbody collisions disabled: {!xRb.detectCollisions}");
    }
}

void EnablePlayerCollisions()
{
    Debug.Log("--- ENABLING Player Systems ---");

    // Enable XR Movement, MoveProvider, CharacterController, Capsule, Rigid Body
    if (xrCC != null)
    {
        xrCC.height = ccHeight;
        xrCC.radius = ccRadius;
        xrCC.center = ccCenter;
        xrCC.stepOffset = ccStepOffset;

        xrCC.enabled = true;
        Debug.Log($"CharacterController enabled: {xrCC.enabled}");
    }

    if (xCapsule != null)
    {
        xCapsule.enabled = true;
        Debug.Log($"CapsuleCollider enabled: {xCapsule.enabled}");
    }

    if (xRb != null)
    {
        xRb.detectCollisions = true;
        Debug.Log($"Rigidbody collisions enabled: {xRb.detectCollisions}");
    }

    if (xrMovement != null)
    {
        xrMovement.enabled = true;
        Debug.Log($"Movement script enabled: {xrMovement.enabled}");
    }
    if (xrDynamicMoveProvider != null)
    {
        xrDynamicMoveProvider.enabled = true;
        Debug.Log($"DynamicMoveProvider enabled: {xrDynamicMoveProvider.enabled}");
    }
}
}
```